Práctica 3: Diseño de estructuras de datos

1. Objetivos de esta práctica

- **Diseñar estructuras de datos.** Se diseñará una pila de datos junto con una colección de funciones básicas para trabajar con ellas.
- **Diseñar algoritmos recursivos**. Se programarán varios algoritmos recursivos que trabajan sobre la estructura de datos definida.

2. Implementación del tipo de dato `PilaEnt`

2.1. El tipo de datos `PilaEnt` y sus funciones básicas

Asumamos que hemos de desarrollar diversos programas que trabajan con pilas de números enteros. Para ello, podemos definir un tipo de dato denominado **`PilaEnt`**, que almacene datos de tipo **`int`**, y una reducida colección de funciones básicas que permitan definir el contenido inicial de una pila, añadir y eliminar datos de ella y consultar su estado. **La implementación de estas funciones no requerirá ni bucles ni recursividad**. Estas funciones se resumen a continuación:

- **`vaciar`**(p): tras ser ejecutada, la pila **`p`** no almacena ningún dato. Esta función permite empezar a trabajar con una pila **`p`** vacía.
- **`apilar`**(p,nuevo): modifica la pila **`p`** añadiéndole el dato entero **`nuevo`**, que se sitúa en su cima, como su dato más reciente.
- **`desapilar`**(p): modifica la pila **`p`**, retirando de **`p`** su dato cima, es decir, su dato más reciente.
- **`cima`**(p): devuelve un valor igual al dato más reciente de la pila **`p`**. Esta función no altera el contenido de la pila.
- **`estaVacia`**(p): devuelve un valor cierto si y sólo si la pila **`p`** no contiene en ese momento ningún dato. Esta función no altera el contenido de la pila.
- **`estaLlena`**(p): devuelve un valor cierto si y sólo si la pila **`p`** está llena por almacenar un número de datos igual a su capacidad máxima. Esta función no altera el contenido de la pila.

Se ha de crear un nuevo directorio, **`programacion2/funciones`**, que alojará diversos ficheros en los que se definan las estructuras de datos y el código de funciones para trabajar con dichas estructuras. En esta carpeta estarán los siguientes ficheros:

CC BY NC SA

- **pilaEnt.hpp**: tendrá que incluir la definición del tipo **PilaEnt** y la interfaz de las funciones básicas mencionadas anteriorment.
- **pilaEnt.cpp**: tendrá que incluir la implementación de las funciones básicas.

El esquema del fichero **pilaEnt.hpp**, mostrado en el Listado 1, se encuentra disponible como parte del código de apoyo proporcionado en Moodle.

Cada alumno debe completar la definición del tipo de dato **PilaEnt** en **pilaEnt.hpp**, e implementar en C++ las seis funciones básicas en el fichero **pilaEnt.cpp**.

```
#ifndef PILAENT_HPP
#define PILAENT_HPP
// Capacidad máxima de almacenamiento de datos en una pila
const int DIM = 120;         // Redefinir su valor en caso necesario

// Un dato de tipo PilaEnt representa una pila de datos enteros.
// Los datos son gestionados según una política LIFO,
// es decir el último en ser apilado será el primero en ser desapilado.
struct PilaEnt {
                            //completar la estructura de datos
};

//  Notación empleada en la especificación de funciones:
//    p = [] la pila está vacía, es decir, almacena 0 datos
//    p = [d_1, d_2, ..., d_K] la pila almacena K datos. El más antiguo
//        es el dato d_1, el segundo más antiguo el dato d_2, etc.,
//        y el más reciente es el dato d_K

// Pre: ---
// Post: p = []
void vaciar(PilaEnt &p);

// Pre: p = [d_1, d_2, ..., d_K] ∧ 0 ≤ K < DIM
// Post: p = [d_1, d_2, ..., d_K, nuevo]
void apilar(PilaEnt &p, const int nuevo);

// Pre: p = [d_1, d_2, ..., d_(K-1), d_K] ∧ 0 < K
// Post: p = [d_1, d_2, ..., d_(K-1)]
void desapilar(PilaEnt &p);

// Pre: p = [d_1, d_2, ..., d_(K-1), d_K] ∧ 0 < K
// Post: cima(p) = d_K
int cima(const PilaEnt &p);

// Pre: ---
// Post: estaVacia(p) = (p = [])
bool estaVacia(const PilaEnt &p);

// Pre: p = [d_1, d_2, ..., d_K]
// Post: estaLlena(p) = (K = DIM)
bool estaLlena(const PilaEnt &p);
#endif
```

Listado 1: Definición del tipo **PilaEnt** y sus funciones básicas (fichero **pilaEnt.hpp**).

CC BY NC SA

2.2. Diseño recursivo de funciones para trabajar con pilas de enteros

Se debe diseñar un módulo de biblioteca denominado **funcionesPilaEnt** que incluya en su fichero de especificación, **funcionesPilaEnt.hpp**, las funciones que se especifican formalmente en el Listado 2. Los ficheros de interfaz e implementación del módulo (**funcionesPilaEnt.hpp** y **funcionesPilaEnt.cpp**, respectivamente) se ubicarán en el directorio (carpeta) **programacion2/practica3**.

Estas funciones trabajan con pilas de datos enteros definidas a partir del tipo **PilaEnt** y resuelven problemas de un nivel de abstracción superior al de las funciones básicas dedicadas a gestionar directamente las pilas, desarrolladas en la primera tarea y contenidas en el fichero **pilaEnt.hpp**.

En el diseño de estas seis nuevas funciones se ha de tener en cuenta lo siguiente:

- El trabajo con una pila ha de ser independiente de la representación de esta, es decir, estas funciones no deben acceder a los campos del struct **PilaEnt**.
- Ninguna de estas funciones puede contener bucles y solo puede llamarse a sí misma y a las funciones declaradas en **pilaEnt.hpp**, véase Listado 1.
- No se debe hacer uso de estructuras de datos auxiliares en las que almacenar temporalmente el contenido de las pilas de datos con las que se trabaje. Es decir, no se pueden definir otras pilas, vectores, ficheros, etc.

Los ficheros **funcionesPilaEnt.hpp** y **funcionesPilaEnt.cpp** se ubicarán en la carpeta **programación2/practica3**. El fichero de interfaz, **funcionesPilaEnt.hpp**, se encuentra disponible como parte del código de apoyo proporcionado en Moodle.

```
#ifndef FUNCIONES_HPP
#define FUNCIONES_HPP

#include <iostream>
#include <iomanip>

// Tipo PilaEnt y operaciones básicas para el trabajo con pilas de datos
#include "pilaEnt.hpp"

// * Notación empleada en las especificaciones:
// p = [] La pila p está vacía, es decir, almacena 0 datos
// p = [d_1, d_2, ..., d_K] la pila p almacena K datos. El más antiguo
//     es el dato d_1, el segundo más antiguo el dato d_2, etc.,
//     y el más reciente es el dato d_K

// * Pre: p = [d_1, d_2, ..., d_K] ∧ 0 ≤ K
// * Post: p = [d_1, d_2, ..., d_K] ∧ numDatos(p) = K
int numDatos(PilaEnt &p);

// Pre: p = [d_1, d_2, ..., d_K] ∧ 0 ≤ K ∧ anchura ≥ 1
// Post: p = [d_1, d_2, ..., d_K] y presenta por pantalla un listado de
//       los datos apilados en la pila comenzando por la cima, d_K, y acabando
//       por el del fondo de la pila, d_1. Cada dato lo escribe en una línea,
```


CC BY NC SA

```
//      empleando anchura caracteres y alineado a la derecha. Cada dato es
//      precedido por el carácter '|' y es seguido por los caracteres ' ' y
//      '|', tal como se ilustra a continuación. Tras el último dato se
//      presenta una linea de la forma "+--...--+", seguida por una línea
//      en blanco:
//       | d_K |
//       | ..  |
//       | d_2 |
//       | d_1 |
//       +--+
void mostrar(PilaEnt &p, const int anchura=3);

// Pre: p = [d_1, d_2, ..., d_K] ∧ 0 ≤ K ∧ 1 ≤ anchura
// Post: p = [d_1, d_2, ..., d_K] y presenta por pantalla un listado de los
//      datos apilados en la pila comenzando por el del fondo de la pila, d_1,
//      y acabando por el de la cima de la pila, d_K. Cada dato lo escribe
//      en una línea, empleando anchura caracteres y alineado a la derecha.
//      Cada dato es precedido por el carácter '|' y es seguido por los
//      caracteres ' ' y '|', tal como se ilustra a continuación. Antes
//      del primer dato se presenta una linea de la forma "+--...--+":
//       +--+
//       | d_1 |
//       | d_2 |
//       | ..  |
//       | d_K |
void mostrarInvertida(PilaEnt &p, const int anchura=3);

// * Pre: p = [d_1, d_2, ..., d_K] ∧ 0 ≤ K
// * Post: p = [d_2, ..., d_K]
void eliminarFondo(PilaEnt &p);


// * Pre: p = [d_1, ..., d_(K-i), d_(K-i+1), d_(K-i+2), ..., d_K] ∧ 0 ≤ K ∧ 1 ≤ i ≤ K
// * Post: p = [d_1, ..., d_(K-i), d_(K-i+2), ..., d_K]
void eliminar(PilaEnt &p, const int i);

// * Pre: p = [d_1, d_2, ..., d_K] ∧ 0 ≤ K
// * Post: p = [nuevo, d_1, d_2, ..., d_K]
void insertarEnFondo(PilaEnt &pila, const int nuevo);

#endif
```

Listado 2: Fichero de interfaz **funcionesPilaEnt.hpp**.

2.2.1. Desarrollo de programas de pruebas de las funciones anteriores

Cada alumno debe ejecutar algún programa para probar el correcto funcionamiento de las diferentes funciones definidas en los apartados anteriores. Estos programas se ubicarán en el directorio (carpeta) **programacion2/practica3**.

Para ilustrar una forma sencilla de diseñar programas de prueba, se proporciona el programa **pruebasPila.cpp** como parte del código de apoyo en Moodle.

CC BY NC SA

2.2.2. Compilación y ejecución del programa de pruebas

El programa de pruebas que se acaba de mencionar consta únicamente de un módulo principal cuyo código se encuentra en el fichero **pruebasPila.cpp**. Este programa depende de las funciones definidas en el módulo **funcionesPilaEnt**, es decir en el código de **pruebasPila.cpp** se ha incluido una cláusula **#include** para la inserción del código del fichero **funcionesPilaEnt.hpp**. De forma similar, el fichero de interfaz **funcionesPilaEnt.hpp** depende de las funciones definidas en el módulo **pilaEnt**.

Tras situarnos en el directorio **programacion2/practica3**, generamos un ejecutable del programa principal **pruebasPila.cpp** considerando las dependencias mencionadas anteriormente.

Es decir, generamos primero los códigos objeto de **pilaEnt.cpp** y **funcionesPilaEnt.cpp** y luego compilamos el programa de prueba y enlazamos los códigos objeto:

```
$ cd $HOME/programacion2/practica3
$ g++ ../funciones/pilaEnt.cpp -I ../funciones -c -o pilaEnt.o -std=c++11
$ g++ funcionesPilaEnt.cpp -I ../funciones -c -o funcionesPilaEnt.o -std=c++11
$ g++ pruebasPila.cpp funcionesPilaEnt.o pilaEnt.o -I ../funciones -o pruebasPila -std=c++11
```

Como resultado se obtiene el fichero **pruebasPila** que puede ser ejecutado cuantas veces se desee.

```
$ ./pruebasPila
  ...mostrará los resultados de ejecutar el pograma anterior
```

En el material de apoyo, se proporciona un fichero **Makefile** que permite compilar el programa **pruebasPila** mediante la orden:

```
$ make
```

3. Resultados del trabajo desarrollado en las tres primeras prácticas

Como resultado de las tres primeras prácticas, cada alumno dispondrá en su cuenta de un directorio (carpeta) denominado **programacion2** dentro del cual se encontrarán los directorios (carpetas) y ficheros que se detallan a continuación.

- Carpeta **programacion2/funciones** con los siguientes ficheros: **pilaEnt.hpp** y **pilaEnt.cpp**
- Carpeta **programacion2/practica1**, con los siguientes ficheros: **tiempoReaccion.cpp**, **generarTabla.cpp**, **genNum.hpp**, **genNum.cpp** y **aproxPI.cpp**.
- Carpeta **programacion2/practica2** con los siguientes ficheros: **calculos.hpp**, **calculos.cpp**, **pruebasCal.cpp** y **Make_pruebasCal**

CC BY NC SA

- Carpeta **`programacion2/practica3`** con los siguientes ficheros:
  - Ficheros de interfaz y de implementación, **`funcionesPilaEnt.hpp`** y **`funcionesPilaEnt.cpp`**.
  - Fichero **`pruebasPila.cpp`** con el programa de prueba.
  - Fichero **`Makefile`** para compilar el programa de prueba.